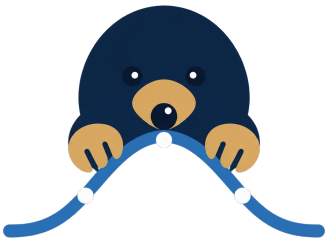

CurveMole User Manual

Preview 0.8.1

Sebastiano Romi

European Laboratory for Non-Linear Spectroscopy (LENS)
University of Florence (UNIFI)

romi@lens.unifi.it

Updated for CurveMole 0.8.1

Contents

CurveMole is a desktop-first, scriptable application for fitting one-dimensional scientific curves. This manual describes the behavior of CurveMole 0.8.1 Preview. The Markdown file is the authoritative source. The LaTeX source and PDF edition are generated automatically from it and must carry the same version number.

> **Preview status.** CurveMole 0.8.1 is suitable for evaluation and controlled testing. It has not yet completed the scientific validation planned for version 1.0.0. Inspect the residuals, parameter correlations, constraints, and exported results before using a fit in research.

Useful links:

- CurveMole repository
- Latest release and desktop downloads
- Issue tracker
- Zenodo record and DOI
- Quick Start

1. About this manual

1.1 Intended audience

This manual is for researchers and technical users who need to fit spectra or other x-y data while retaining control over data preparation, model construction, constraints, uncertainty, and export. Typical data include:

- infrared and Raman spectra;
- powder diffraction patterns;
- kinetic traces;
- chromatographic profiles;
- temperature, pressure, or field-dependent signals;
- any other one-dimensional curve that can be represented by numeric x and y columns.

No programming knowledge is required for the graphical workflow. Separate chapters cover the command-line interface, YAML workflows, Python API, formula builder, and plugins.

1.2 Version policy for the manual

The version in the title and in the line at the top of this file identifies the CurveMole release whose behavior is documented. It is not an independent document version. For example, a manual marked 0.8.1 describes CurveMole 0.8.1.

The release automation performs four checks:

1. the application version and both manual version declarations must agree;
2. all local links in the Markdown source must resolve;
3. Pandoc must generate a standalone LaTeX document without warnings;
4. XeLaTeX must compile the generated source into a valid PDF.

The release assets use versioned names so that a downloaded manual can always be associated with the correct executable.

1.3 Terminology and notation

Menu commands are written as **File > Import data**. Buttons and field names are shown in **bold**. Commands, filenames, identifiers, and parameter paths use `monospace` text.

CurveMole uses the following residual convention in displays and exports:

$$r_i = y_i - f(x_i).$$

The optimizer minimizes the opposite sign, $f(x_i) - y_i$, after applying uncertainty and spectrum scaling. The sign has no effect on a least-squares minimum but matters when reading exported residuals.

1.4 Scientific responsibility

CurveMole can optimize a mathematically defined model, but it cannot decide whether that model is physically appropriate. A small residual or a high descriptive R^2 does not by itself establish uniqueness, chemical meaning, or predictive validity. In particular:

- do not add components solely to improve a scalar fit statistic;
- use bounds only when they have a defensible meaning;
- inspect correlations and active bounds;
- check whether residuals contain structure;
- report the exact CurveMole version, model, constraints, weighting, and uncertainty method used.

2. Core concepts

CurveMole keeps the scientific engine independent from the graphical interface. The desktop application, command-line interface, YAML workflows, and Python API all operate on the same domain objects.

2.1 Project

A project is the complete analysis workspace. It can contain data, models, masks, transformations, fit results, custom formulas, history, and interface state. Projects are saved as `.fitproj` files.

2.2 Series and curves

A **series** groups related curves. A **curve** contains one x array and one y array, with optional `sigma_x`, `sigma_y`, or point weights. Importing multiple y columns from a common file produces multiple curves in one imported series.

Original numeric arrays are stored as immutable 64-bit floating-point values. Data Calculator operations create reversible transformations instead of overwriting the original arrays.

2.3 Model, component, and parameter

Each curve has its own model. A model is an ordered sequence of components such as a background and several peaks. Every component has:

- a function, for example `gaussian` or `linear`;
- a user-facing name;
- a composition operator;
- a set of parameters;
- an enabled or disabled state;
- optional metadata, such as polynomial order or spline x nodes.

A parameter has a current value, optional lower and upper bounds, a fixed state, and an optional expression link. Standard errors and confidence limits are populated after a successful fit when they are available.

2.4 Active, selected, and visible curves

These states are deliberately distinct:

- **Active curve:** the single curve whose model and handles are being edited.
- **Selected curves:** one or more curves chosen for multi-curve operations.
- **Visible curves:** curves drawn in Overlay or Waterfall display modes.

Activating a curve does not automatically select every visible curve. Before a fit, mask transfer, or multi-curve calculation, check the target shown in the relevant dialog.

2.5 Fit plan and result

A fit plan specifies the curves, fit mode, solver settings, per-spectrum weights, and whether numerical contributions should be equalized. A fit result records the optimized parameters, statistics, residual arrays, covariance, correlation, warnings, timestamp, and solver status.

3. Installation and first launch

3.1 Reference requirements

The current reference configuration is:

Item	Minimum or supported target
Processor	x86-64, 4 CPU cores recommended
Memory	8 GB RAM
Storage	SSD recommended
Display	1920 x 1080 recommended
Windows	Windows 10 22H2 legacy compatibility or Windows 11, 64-bit
macOS	macOS 13 Ventura or newer, Intel or Apple Silicon
Linux	Ubuntu 22.04 LTS, Debian 12, or a comparable newer distribution
Python	Python 3.12 or newer for Python installations

A dedicated GPU is not required. Large multi-curve fits and resampling analyses may require more memory and time than the reference configuration.

3.2 Desktop packages

Open the latest release and select the package for your system.

Platform	Release asset	Normal launch method
Linux x86-64	`CurveMole-VERSION-linux-x86_64.AppImage`	Mark executable and run
Windows x86-64	`CurveMole-VERSION-windows-x86_64.exe`	Double-click the executable
macOS Apple Silicon	`CurveMole-VERSION-macos-arm64.dmg`	Open the DMG and launch the app
macOS Intel	`CurveMole-VERSION-macos-x86_64.dmg`	Open the DMG and launch the app

Windows and macOS packages are currently unsigned. The operating system may show a warning on first launch. Download only from the official GitHub release page and verify the checksum if the application will be used for research.

3.3 Linux AppImage

In a terminal opened in the download directory:

```
chmod +x CurveMole-0.8.1-linux-x86_64.AppImage
./CurveMole-0.8.1-linux-x86_64.AppImage
```

If the system cannot mount AppImages through FUSE, use extraction mode:

```
APPIMAGE_EXTRACT_AND_RUN=1 ./CurveMole-0.8.1-linux-x86_64.AppImage
```

Some minimal Linux installations may need graphical runtime libraries supplied by the distribution. On Debian or Ubuntu, the commonly required packages are:

```
sudo apt install libegl1 libgl1 libxkbcommon-x11-0 libxcb-cursor0
```

3.4 Verifying SHA-256 checksums

Every release includes `SHA256SUMS.txt`. On Linux:

```
sha256sum -c SHA256SUMS.txt --ignore-missing
```

On macOS:

```
shasum -a 256 CurveMole-0.8.1-macos-arm64.dmg
```

On Windows PowerShell:

```
Get-FileHash .\CurveMole-0.8.1-windows-x86_64.exe -Algorithm SHA256
```

Compare the reported value with the corresponding line in `SHA256SUMS.txt`.

3.5 Installing the Python package

Use an isolated environment. From a downloaded wheel:

```
python3 -m venv ~/.venv/curvemole
source ~/.venv/curvemole/bin/activate
python -m pip install ./curvemole-0.8.1-py3-none-any.whl
curvemole gui
```

On Windows PowerShell, activation is:

```
py -m venv $env:USERPROFILE\.venv\curvemole
& $env:USERPROFILE\.venv\curvemole\Scripts\Activate.ps1
python -m pip install .\curvemole-0.8.1-py3-none-any.whl
curvemole gui
```

3.6 Running from a source checkout

Clone the repository and install its locked development environment:

```
git clone https://github.com/SebRoLENS/curvemole.git
cd curvemole
uv sync --locked --group dev
uv run curvemole gui
```

For an editable installation using `pip`:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e .
curvemole gui
```

3.7 Confirming the installed version

```
curvemole --version
```

The About dialog also displays the version. A project stores the application version that created it, and exports include the application version in machine-readable metadata.

4. A complete first fit

This tutorial uses `examples/gaussian.csv` from the source archive. It contains x, y, and `sigma_y` columns.

4.1 Import the example

1. Start CurveMole.
2. Choose **File > Import data** or press **Ctrl+I**.
3. Select `examples/gaussian.csv`.
4. In the mapping dialog, confirm comma delimiter, decimal point, and header.
5. Select x as **X column** and check y under **Y column(s)**.
6. Select `sigma_y` as the uncertainty type and column.
7. Press **OK**.

The curve appears in the left tree and becomes active. The central plot displays the data, and the model panel on the right identifies the active curve.

4.2 Add a background

1. Press the **+** button in **Model and parameters**, or press **Ctrl++**.
2. Select **Constant background**.
3. Keep composition set to `add`.
4. Press **OK**.
5. In the parameter table, enter an initial `offset` near the visual baseline.

4.3 Add a Gaussian peak graphically

1. Add another component and select **Gaussian**.
2. Press **OK**. The pointer changes to a crosshair.
3. Click at the approximate peak center.
4. Keep the mouse button pressed and drag horizontally from the center to either half-height edge.
5. Release the mouse button.

CurveMole uses twice the dragged distance as the initial FWHM. It estimates a positive peak height from the data minus the existing model, converts this height to the area parameterization, and adds the component as one undoable action.

If placement is incorrect, press **Ctrl+Z** and repeat. Press **Esc** during placement to cancel without adding the component.

4.4 Adjust the model before fitting

Select the Gaussian in the component list. Three green handles appear:

- drag the central target to change center and height;
- drag either vertical side handle to change FWHM;
- edit exact values or bounds in the parameter table.

For this example, a positive lower bound on `sigma` already exists intrinsically. You may enter a tighter scientifically motivated range, but it is not required.

4.5 Mask an interval if needed

To exclude a region directly, drag over it with the right mouse button. This works without enabling Mask mode. The current **Mask/Unmask** operation and **Target** selection are respected.

Alternatively:

1. enable **Mask** above the plot;
2. left-click to mask the nearest point, or left-drag an interval;
3. change the operation to **Unmask** to restore points;
4. disable Mask mode when finished.

The example does not normally require masking.

4.6 Run the fit

1. Press **F5** or choose **Fit > Fit**.
2. Use **Single / independent** mode.
3. Check that the example curve is enabled in the table.
4. Keep **Local constrained least squares** and `linear` loss.
5. Press **OK**.

After a successful fit, CurveMole commits the optimized parameter values back into the displayed model and immediately redraws the model sum, individual components, and residual panel. Parameter standard errors appear in the `±1 sigma` column when covariance is available. Open **View > Diagnostics** to inspect residual warnings.

4.7 Save and export

Save the complete workspace with **File > Save project as**. Use a `.fitproj` extension.

Then choose **File > Export analysis bundle**. Select a destination and choose both how to save and **What to export?**. The default selection writes only `fit_results.csv`, which associates every curve with its model functions and lists each parameter together with its fitted value and standard error. Data tables, plots, reports, JSON, reusable models, project copies, uncertainty matrices, and diagnostics are optional.

5. Main window and controls

5.1 Series and curves tree

The left dock contains a search field and a tree with **Visible**, **Series / Curve**, and **State** columns.

- Click a curve to activate it.
- Use Ctrl-click or Shift-click for a multi-selection, or use **Select all** and **Deselect all** above the tree.
- Use **Remove selected** to delete one or more accidentally imported curves. Removal is undoable.
- Use **New series** to create an empty named series. Empty series are retained when the project is saved.
- Right-click one or more selected spectra and choose **Move selected to series** to reorganize them.
- Right-click selected spectra and choose **Move selected up/down** to change their order inside a series. Multiple adjacent spectra move as a stable group.
- Right-click a series and choose **Merge series into** to append all its spectra to another series and remove the source series.
- Moving, merging, creating, and reordering series are all Undo/Redo operations and do not invalidate fitted parameters.
- Use the visibility checkbox to include or exclude it from Overlay and Waterfall displays.
- Double-click an editable name to rename a series or curve. Series names must remain unique.
- Right-click a **curve** and choose **Choose spectrum colour...** to set its colour. Red is reserved for the fitted Model sum and cannot be assigned to a spectrum.
- Right-click a **series** and choose **Series palette** to recolour the complete series with one of the built-in non-red palettes. Palette and individual colours are saved in the project.
- Use the search field to filter curves by name without removing them.

Curve states are:

State	Meaning
Not fitted	No successful fit is associated with the current curve state
Ready	The curve is prepared for fitting
Running	A fit involving the curve is in progress
Fitted	The last fit completed successfully
Modified/outdated	Data or model changed after the last successful fit
Failed	The latest fit attempt failed

5.2 Plot workspace

The central workspace contains the data/model plot and an optional residual plot. Controls above it provide:

- **Display:** Single, Overlay, or Waterfall;
- **X offset** and **Y offset:** display-only Waterfall spacing;
- **Mask:** explicit masking mode;
- **Mask/Unmask:** operation to apply;

- **Target:** Active, Selected, or All visible curves;
- **Residuals:** show or hide the linked residual panel.

Offsets never modify data and are removed when converting pointer positions back to scientific coordinates. Fitting and export use the original numeric coordinates, not screen offsets.

The coordinate readout displays the pointer coordinate and the nearest finite point from the active curve. Rendering may be downsampled by the plotting library for speed, but fitting and export use all usable points.

Model functions receive systematic names such as **Voigt1**, **Voigt2**, and **Gaussian1**. Their labels are shown above each function maximum by default and are shifted vertically when necessary to avoid overlap. Right-click the main plot and toggle **Show component labels** to hide or show these labels. After every completed fit, CurveMole explicitly applies the returned optimized parameters to the displayed model, redraws and auto-ranges the plot so the newly fitted curves are visible immediately. The **Model sum is always red**; imported spectra and selectable series palettes exclude red so a data curve can never be confused with the fitted sum.

5.3 Navigation and axes

Normal plot navigation is available when neither placement nor Mask mode is active. Use **View > Axes** for:

- automatic range;
- logarithmic x or y;
- reversed x or y;
- locked plot view.

Reversing an axis is only a view setting. Logarithmic axes do not transform the data used by the fit. Values incompatible with a log display may not be visible even though they remain stored.

5.4 Model and parameters dock

The right dock lists components for the active curve. Component order matters because composition operators are evaluated sequentially.

The controls below the list are:

Control	Action
+	Add a component
Duplicate	Duplicate the selected component and its settings
Up / Down	Change component order
Delete	Delete the component after confirmation
Copy fit	Copy selected model information to other curves

Uncheck a component in the list to disable it without deleting it. Disabled components are not evaluated or fitted.

5.5 Dockable tools

The **View** and **Tools** menus show or hide these docks:

- **Worksheet:** read-only view of current x, y, uncertainty, weight, and mask state;
- **Diagnostics:** residual summary and warnings;
- **Log:** operational messages and errors;
- **Data Calculator:** reversible scalar and curve-to-curve transformations;
- **Function Builder:** safe mathematical custom functions;
- **Uncertainty Analysis:** Monte Carlo, bootstrap, and profile calculations.

Docks can be moved, tabbed, resized, or floated. CurveMole remembers window geometry, dock arrangement, and theme. Use **View > Reset layout** to restore the standard layout.

5.6 Themes

View > Theme provides System, Light, and Dark themes. Theme choice affects only presentation. It does not change plot data, exported numeric values, or fitting.

5.7 Drag and drop

Drop a `.fitproj` file onto the window to open it. Drop TXT, DAT, CSV, or TSV files to begin import. If a drop contains both a project and data files, the first project is opened.

6. Importing data

6.1 Supported text formats

The graphical importer accepts `.txt`, `.dat`, `.csv`, and `.tsv`. Files must contain at least two columns and at least two rows after parsing.

CurveMole detects common delimiters:

- whitespace;
- comma;
- semicolon;
- tab;
- vertical bar.

It also proposes decimal point or decimal comma and whether the first data row is a header. Always inspect the preview before accepting the mapping.

6.2 Comments, blank lines, and encoding

Blank lines are ignored. Lines whose first non-space character is `#`, `;`, `%`, or `!` are treated as comments by the default importer. UTF-8 with an optional byte order mark is the default encoding.

The semicolon can therefore be ambiguous: a line beginning with semicolon is a comment, while semicolons inside table rows can be delimiters. Check the preview for locale-specific exports.

6.3 Column mapping

Choose exactly one x column and one or more y columns. Each selected y column becomes a separate curve sharing the selected x column. Use **Select all Y columns** or **Deselect all Y columns** when importing tables with many signals. The Python API additionally supports explicit x-y column pairs.

Optional columns are:

- `sigma_x`;
- `sigma_y`;
- generic weight;
- variance;
- inverse variance.

Only one of `sigma_y`, generic weight, variance, or inverse variance can be selected for a given import. Variance is converted to `sigma_y` by square root. `sigma_x` is stored in the project but is not used by the version 0.8.1 optimizer.

6.4 Batch import

When importing multiple files, **Apply this mapping to all files in this batch** reuses the first file's parsing and column mapping. Enable it only if all files truly share a layout. CurveMole does not silently guess a new mapping for an incompatible file in the same batch.

6.5 Invalid numeric cells

Non-numeric cells are converted to missing numeric values. Rows are retained in the stored arrays but excluded from fitting when x or y is not finite. A row is also excluded if:

- `sigma_y` is non-finite or not strictly positive;
- a point weight is non-finite or negative;
- it is masked;
- it lies outside active fit ranges defined through the core API.

CurveMole does not reorder rows, average duplicate x values, or interpolate missing values during import.

6.6 Uncertainties and weights

If absolute `sigma_y` values are supplied, point residuals are scaled as:

$$r_i^{(w)} = \frac{f(x_i) - y_i}{\sigma_{y,i}}.$$

If inverse variances $w_i = 1/\sigma_i^2$ are supplied, CurveMole multiplies residuals by $\sqrt{w_i}$. A generic weight column directly multiplies residuals by its value. This distinction is intentional and is recorded in the curve metadata.

Zero generic weight is allowed and gives a point zero numerical influence. A zero or negative `sigma_y` is invalid and excludes the row.

7. Curves, masks, and transformations

7.1 Immutable originals

CurveMole preserves imported x, y, uncertainty, and weight arrays. Displayed working arrays are reconstructed by applying a transformation history to the originals. This design makes data preparation reversible and auditable.

7.2 Masking points and intervals

Masked points remain stored but are excluded from fitting. They are drawn as faded markers, and masked intervals are shaded.

There are two interaction styles:

1. Enable **Mask**, then left-click a point or left-drag an interval.
2. At any time, right-drag an interval without enabling Mask mode.

Both use the current operation and target. Select **Unmask** before dragging to restore an interval.

7.3 Mask targets

Target	Effect
Active	Changes only the active curve
Selected	Changes every selected curve
All visible	Changes every currently visible curve

For interval masks, the same numeric x interval is applied to every target. For a single-point mask transferred from the active curve, CurveMole locates the nearest x value on other targets and applies it only if it lies within the configured tolerance.

Set this tolerance through **Data > Mask transfer tolerance**. A value of zero requires an exact matching x value on transferred point masks.

7.4 Undo and mask history

A graphical mask action is placed on the application Undo stack. Press **Ctrl+Z** to undo it or **Ctrl+Shift+Z** on platforms that use that standard Redo binding. The project also stores mask arrays and numeric mask intervals.

7.5 Data Calculator

Open **Tools > Data Calculator**. The following scalar operations are available:

- add to or subtract from y;
- multiply or divide y;
- shift x;
- scale x;
- normalize y by maximum absolute value;
- normalize y by signed integrated area.

Choose Active curve, Selected curves, or Entire series as the target. Normalization also scales `sigma_y` consistently. Dividing by zero and normalizing a zero or invalid signal are rejected.

7.6 Curve-to-curve calculations

The calculator can add, subtract, multiply, or divide by another curve. The operand is aligned to the target x grid by:

- linear interpolation;
- nearest-neighbor interpolation;
- cubic spline interpolation.

Extrapolation is disabled by default. Outside the operand range, aligned values become missing and will therefore be excluded from later fitting. Enabling extrapolation is an explicit advanced choice. Cubic interpolation requires at least four unique finite operand x values.

The aligned operand array is stored with the transformation so that reopening the project does not depend on a later change to the source curve.

7.7 Restoring original data

Restore original data removes all currently applied transformations from the selected target while retaining them on the redo stack. The imported arrays are not re-read from disk. The whole action is also integrated with application Undo.

7.8 Worksheet

The Worksheet is a read-only inspection view. It shows up to 100,000 rows for the active curve with columns for x, y, `sigma_y`, weight, and mask state. It is not a spreadsheet editor and does not modify source arrays.

8. Building models

All entries in the function library are fitting functions. Constant, linear, polynomial, cubic-spline, peak-shaped, and custom formulas are not separated into a special background-function class. Background is instead a property of a model component. Select a component in **Model and parameters** and enable **Mark as background** when that component represents the experimental baseline or background.

Data > Subtract background... uses these component-level marks. If no component is marked, CurveMole first asks which model functions should be designated as background. If background components already exist, it asks which of the marked components should be subtracted. The subtraction uses the current resolved/fitted parameter values, is reversible with Undo, applies over the complete data array, and disables the subtracted components afterwards to prevent double-counting.

During graphical cubic-spline placement, nodes may be placed anywhere in plot coordinates, including outside the x/y extent of the measured data. Adding nodes does not autorange the graph. Left-drag continues to pan and the mouse wheel continues to zoom while placement is active.

8.1 Component composition

Components are evaluated in list order. Available operators are:

Operator	Operation
`add`	Add the component array to the accumulated model
`subtract`	Subtract the component array
`multiply`	Multiply the accumulated model by the component array
`divide`	Divide the accumulated model by the component array
`convolve`	Convolve using FFT and scale by median x spacing

The first enabled component can use only `add` or `subtract`; multiplication, division, or convolution requires an existing accumulated model. Additive peak plus background models should normally use `add` for every component.

Division by a component that reaches zero and other non-finite model evaluations are rejected by the fitter.

8.2 Built-in peak parameterization

Every built-in peak uses signed integrated `area`, not peak height, as its intensity parameter. Negative areas produce negative peaks. This common parameterization makes areas comparable between different line shapes.

Gaussian

Parameters: `area`, `center`, `sigma`.

$$G(x) = \frac{A}{\sigma\sqrt{2\pi}} \exp\left[-\frac{1}{2}\left(\frac{x - x_0}{\sigma}\right)^2\right].$$

The derived FWHM is $2.354820045\,\sigma$. `sigma` is intrinsically positive.

Lorentzian

Parameters: `area`, `center`, `gamma`.

$$L(x) = \frac{A\gamma}{\pi[(x - x_0)^2 + \gamma^2]}.$$

`gamma` is the half width at half maximum, so FWHM is 2γ. `gamma` is intrinsically positive.

Voigt

Parameters: `area`, `center`, `sigma`, `gamma`.

The Voigt component is the area-normalized convolution of a Gaussian with standard deviation `sigma` and a Lorentzian with HWHM `gamma`. CurveMole evaluates it with the SciPy Voigt profile. Both width parameters are intrinsically positive.

The displayed FWHM is the approximation:

$$\text{FWHM} \approx 0.5346(2\gamma) + \sqrt{0.2166(2\gamma)^2 + (2.354820045\,\sigma)^2}.$$

Pseudo-Voigt

Parameters: `area`, `center`, `fwhm`, `eta`.

The profile is an area-normalized linear mixture with a common FWHM:

$$pV(x) = A[(1 - \eta)G_{\text{unit}}(x) + \eta L_{\text{unit}}(x)].$$

`fwhm` is intrinsically positive and `eta` is intrinsically bounded to $[0, 1]$. `eta=0` is Gaussian and `eta=1` is Lorentzian.

8.3 Built-in backgrounds

Constant

Parameter: `offset`.

$$B(x) = c.$$

Linear

Parameters: `intercept`, `slope`.

$$B(x) = c_0 + c_1 x.$$

Polynomial

Choose an order from 0 to 50 when adding the component. Parameters are `c0`, `c1`, and so on through the chosen order:

$$B(x) = \sum_{k=0}^{n} c_k x^k.$$

High polynomial orders are numerically sensitive when x values are large or poorly scaled. Prefer a low order or rescale x with the Data Calculator when scientifically appropriate.

Cubic spline

Spline x nodes are fixed metadata. Their y values (`y0`, `y1`, and so on) are normal parameters that may be fitted, fixed, bounded, or linked. Two nodes produce linear interpolation. Three or more nodes produce a natural cubic spline, with extrapolation outside the node range.

8.4 Adding a peak with the pointer

After choosing a peak and pressing **OK**:

1. click the intended center;
2. drag horizontally to either half-height edge;
3. release to set the initial full width.

A green region previews the width. If no horizontal width is selected, CurveMole uses a default based on 5 percent of the x span, local point spacing, and floating point precision.

For built-in peaks, CurveMole converts the graphical height and FWHM into the native area and width parameters. For custom peak formulas, it recognizes conventional parameter names such as `center`, `x0`, `fwhm`, `sigma`, `gamma`, `area`, `amplitude`, or `height` when possible.

8.5 Adding a spline background with the pointer

After choosing **Cubic-spline background**:

1. left-click background nodes on the plot;
2. inspect the dashed preview, which updates after each click;
3. use **Undo point** if necessary;
4. finish with **Finish** or a right-click after at least two nodes;
5. press **Esc** to cancel the entire placement.

Nodes are sorted by x. Clicking again at effectively the same x updates that node instead of creating a duplicate. Exact duplicate x positions are not permitted.

8.6 Graphical editing after placement

Select a peak component to display its handles. Drag the central target to change center and area-derived height. Drag a side line to change FWHM. For a Voigt peak, changing FWHM graphically scales `sigma` and `gamma` by the same ratio, preserving their current relative contribution.

Select a cubic spline to drag its y nodes. Node x positions remain fixed. Exact numeric edits are available in the parameter table.

Fixed values cannot normally be changed by a drag. Hold **Ctrl** while dragging to change the stored value while leaving the parameter fixed. Linked values cannot be dragged because their value is controlled by an expression.

8.7 Automatic peak suggestions

Choose **Model > Find positive peaks**. The dialog can search positive, negative, or both signs. CurveMole:

- subtracts the median as an initial baseline;
- estimates noise through a robust median absolute deviation;
- uses a default prominence of the larger of three noise estimates or 1 percent of the y range;
- estimates width at half prominence;
- sorts suggestions by prominence.

Choose how many suggestions to add. Version 0.8.1 creates Gaussian components from this graphical command. Suggestions are initial estimates, not a scientific decision about the number or identity of peaks.

8.8 Duplicating, disabling, deleting, and reordering

Duplicate a component to create an adjacent copy with a new internal identifier. Disable a component to compare models without losing its settings. Deletion and reordering are undoable. Reordering can materially change models that use operations other than addition or subtraction.

8.9 Copying a fit to other curves

Model > Copy fit copies selected information from the active curve. Use **Select all** or **Deselect all** when choosing many target curves. Options include:

- component structure;
- current or best values;
- bounds and fixed states;
- internal links;
- background components;
- masks;
- fit ranges.

Masks and fit ranges are excluded by default because x grids and excluded regions may not correspond scientifically. When copying masks, point transfer uses the configured x tolerance. Review every target model after copying.

9. Parameters, constraints, and links

9.1 Parameter table

The table columns are:

Column	Meaning
Parameter	Parameter name; linked parameters show a link indicator
Value	Current value or successful fitted value
±1 sigma	Covariance-based standard error when available
Fixed	Exclude the parameter from optimization
Lower	Lower bound; blank means negative infinity
Upper	Upper bound; blank means positive infinity
Link	Graphical **Set link...** control; linked parameters show their source in readable form

Press **Set link...** to choose a source spectrum, component, and parameter. The default relationship is **Equal to source**. Choose **Advanced expression** for relationships such as `2 * ${source} + 1`. Use **Remove link** to make the parameter independent again.

An edit is validated immediately. CurveMole rejects a value outside its bounds, a lower bound above an upper bound, an invalid expression, a missing referenced parameter, a link cycle, or a linked value that violates its own bounds.

9.2 Parameter states

A parameter is exactly one of:

- free;
- fixed;
- lower-bounded;
- upper-bounded;
- interval-bounded;
- linked.

Intrinsic function bounds always remain active. For example, clearing the visible lower cell cannot make a Gaussian `sigma` negative because the function definition sets a positive intrinsic minimum.

9.3 Parameter paths

The canonical path is:

```
curve_id.component_id.parameter_name
```

A link refers to a path inside `${...}`. For example:

```
${curve_ab12.component_cd34.center}
```

or a derived relation:

```
${curve_ab12.component_cd34.center} + 12.5
```

The graphical **Set link...** dialog creates these paths automatically, so normal GUI use does not require knowing curve or component identifiers. The canonical expression is still shown as a tooltip and remains available to advanced workflows, Python code, and machine-readable exports. For a link between different spectra, select both spectra and use **Global simultaneous** mode; CurveMole blocks incompatible fit modes with an explicit message.

9.4 Link evaluation

Links form a directed dependency graph. CurveMole resolves the graph before fitting, detects cycles, and evaluates linked values at each model evaluation. Linked parameters are not independent optimizer variables. If covariance is available, their standard errors are propagated numerically from free parameters.

9.5 Safe expression language

Formulas and links are parsed as a restricted expression tree. They are never passed to unrestricted Python `eval` or `exec`.

Allowed arithmetic includes `+`, `-`, `*`, `/`, power `**`, remainder `%`, unary signs, comparisons, boolean combinations, and conditional expressions. Constants are `pi`, `e`, and `inf`.

Allowed functions are:

```
abs, sqrt, exp, log, log10,
sin, cos, tan, arcsin, arccos, arctan,
sinh, cosh, tanh,
erf, erfc,
minimum, maximum, clip, where, heaviside
```

Attribute access, imports, indexing, comprehensions, arbitrary function calls, file access, and network access are not part of the expression language.

10. Fitting

10.1 Pre-fit checklist

Before pressing **F5**:

1. confirm the active and selected curves; use **Select all** / **Deselect all** when appropriate;
2. inspect invalid and masked points;
3. choose a physically defensible background and peak count;
4. provide reasonable initial values;
5. add justified bounds for poorly determined parameters;
6. inspect links for cycles or unintended dependencies;
7. check that each selected model contains at least one free parameter.

10.2 Fit modes

Single / independent

Each selected curve is fitted as an independent problem. Results are then combined for reporting. Parameter links between different independently fitted curves cannot act as a simultaneous cross-curve constraint; use Global mode for that purpose.

Sequential

Curves are fitted in the listed order. Before fitting the next curve, values from matching components in the previous model are copied where the target parameters are free. Matching uses component order and function type.

If a curve fails, the sequence pauses at that curve. Correct its model manually, then choose **Fit > Continue paused sequence**. CurveMole does not silently skip the failure.

Global simultaneous

All selected residual arrays are concatenated into one optimization problem. Models may differ between curves. Parameter links can connect curves and express shared, offset, scaled, or otherwise related values.

10.3 Per-spectrum weights

The Fit dialog accepts a positive numeric spectrum weight for every selected curve. If s_j is the spectrum weight, all point residuals from spectrum j are multiplied by $\sqrt{s_j}$.

Scale each spectrum to equal numerical contribution additionally divides the residual vector of each curve by the square root of its number of usable points. This prevents a densely sampled curve from dominating only because it contains more rows. It does not normalize signal amplitude or uncertainty and is never enabled silently.

10.4 Local least squares

The default is local nonlinear least squares using SciPy. CurveMole automatically selects:

- Levenberg-Marquardt for an unbounded ordinary least-squares problem;
- trust-region reflective when bounds or a robust loss require it.

The graphical dialog exposes the maximum number of evaluations and confidence level. The Python API and YAML settings additionally expose tolerances, scaling, local method, random seed, and Differential Evolution controls.

10.5 Robust losses

Available losses are:

- `linear` for ordinary least squares;
- `soft_l1`;
- `huber`;
- `cauchy`.

Robust loss can reduce the influence of large residuals, but it is not a substitute for understanding outliers, detector artifacts, or an incomplete model. AIC, AICc, and BIC are reported only for the linear loss. Robust-loss covariance uses a sandwich estimate, and CurveMole recommends explicit resampling.

10.6 Differential Evolution initial search

Choose **Differential Evolution + local refinement** when a local initial estimate is not sufficient. Differential Evolution searches globally, then passes its best point to the local least-squares solver.

Every free parameter must have finite user bounds. CurveMole will not invent search bounds because arbitrary limits can change scientific conclusions. Global search is usually much slower than a well-initialized local fit.

10.7 Cancellation and failed fits

Use **Fit > Cancel running task** to request cancellation. The request is checked during residual evaluation and resampling. The window cannot close while a background task remains active.

If the solver raises an error or does not converge, CurveMole restores the last valid parameter values instead of committing an incomplete optimizer vector. The curve is marked Failed, the error is written to the Log, and the last successful result remains separate from the failed attempt.

10.8 Common causes of failure

- no free parameters;
- fewer than two usable points;
- initial value outside bounds;
- non-finite model values;
- division by zero in a composed model;
- a missing or cyclic link;
- a linked value outside its bounds;
- unbounded parameters with Differential Evolution;
- an underdetermined model;
- inadequate initial values or maximum evaluations.

11. Results, statistics, and diagnostics

11.1 Visual results

The main plot shows data, total model, individual component curves, and residuals. The parameter table shows fitted values and standard errors. A curve state changes to Fitted only after solver success.

The full numeric result is stored in the project and written to exports. Preview 0.8.1 does not yet provide a single comprehensive on-screen results table, so use the analysis bundle for archival inspection.

11.2 Reported global statistics

Let N be the number of usable observations and k the number of independent free parameters. CurveMole reports:

Statistic	Definition or interpretation
Degrees of freedom	$N - k$
RSS	$\sum_i (y_i - f_i)^2$ without point weighting
RMSE	$\sqrt{\mathrm{RSS}/N}$
Chi-square	Sum of squared weighted residuals
Reduced chi-square	Chi-square divided by $N - k$ when positive
Descriptive R^2	$1 - \mathrm{RSS}/\mathrm{TSS}$
AIC	Gaussian ordinary least-squares expression when applicable
AICc	Small-sample correction when $N > k + 1$
BIC	Gaussian ordinary least-squares expression when applicable

The descriptive R^2 can be misleading for baselines, constrained fits, or models without an intercept. It is included as a familiar descriptor, not a universal model quality score.

11.3 Per-curve statistics

Each curve output records the original usable indices, x, observed y, fitted y, residual, weighted residual, N, RSS, RMSE, and descriptive R^2. The global result combines all curves in the plan.

11.4 Covariance and correlation

For ordinary least squares, covariance is derived from the fit Jacobian. If all curves provide `sigma_y`, the uncertainty is treated as absolute by default and the covariance is not rescaled by residual variance. Otherwise residual variance scales the covariance.

CurveMole uses a pseudo-inverse when the Jacobian is rank deficient and reports a warning. Covariance is unavailable when degrees of freedom are not positive. A warning is also generated when at least one free-parameter correlation has absolute value 0.95 or greater.

Symmetric normal intervals are clipped to active bounds. A parameter at a bound is flagged because its true uncertainty is generally asymmetric and not adequately summarized by a symmetric covariance interval.

11.5 Residual diagnostics

Open **View > Diagnostics** after a fit. The summary reports:

- Durbin-Watson statistic;
- number of residuals with absolute standardized value at least 3;
- lag-1 autocorrelation;
- warnings for potential outliers and autocorrelation beyond an approximate 95 percent white-noise band.

The analysis bundle exports the autocorrelation sequence for every fitted curve. A structured residual pattern usually indicates that the model, background, weighting, or data treatment deserves review.

12. Explicit uncertainty analyses

Open **Fit > Uncertainty Analysis** only after a successful fit. These calculations run in the background, can be cancelled, and may be substantially slower than the baseline fit.

12.1 Parametric Monte Carlo

This method generates synthetic y values from the fitted model plus independent normal noise using the imported absolute `sigma_y` array. Every selected curve must therefore provide valid `sigma_y`.

Each synthetic data set is refitted using the local solver. The result records the requested, completed, and failed replicates, seed, free parameter paths, samples, empirical intervals, and failure messages.

12.2 Residual bootstrap

Residuals are centered, sampled independently with replacement, added to the fitted model, and refitted. This assumes residual exchangeability and can be inappropriate when residuals are serially correlated or heteroscedastic.

12.3 Block bootstrap

The block bootstrap resamples contiguous circular residual blocks. Enter a block length or choose Automatic. The automatic estimate uses the first lag at which the absolute residual autocorrelation falls below e^{-1}, subject to practical limits.

Block resampling is often preferable for spectra or kinetic traces with local correlation, but the block length remains a scientific modeling decision.

12.4 Profile likelihood

Choose one independent, non-linked parameter. CurveMole fixes it across a finite grid, refits all remaining free parameters, and compares chi-square with the baseline. The default grid spans approximately three covariance standard errors on either side, or a fallback span when no standard error is available, while respecting bounds.

Preview 0.8.1 uses 31 grid points and a one-parameter chi-square threshold. Failed grid points are counted. A profile interval is more informative than a symmetric standard error near bounds or in nonlinear problems, but grid resolution should be considered when interpreting endpoints.

12.5 Reproducibility and failed replicates

The default seed is 1729 unless changed through programmatic settings. Resampling outputs record configuration and up to the first 100 failure messages. Do not report an empirical interval without also reporting how many replicates completed and failed.

13. Projects, saving, and recovery

13.1 The `.fitproj` format

A `.fitproj` is a ZIP container with:

- a versioned JSON manifest;
- original arrays stored as NumPy `.npy` payloads;
- uncertainty and weight arrays when present;
- mask arrays;
- transformation operands;
- models, links, custom functions, results, history, and UI/export state;
- SHA-256 checksums for every binary payload.

NumPy arrays are loaded with `allow_pickle=False`. CurveMole projects never require executing a Python pickle. The format is documented in project-format.md.

13.2 Atomic saves

CurveMole writes a temporary sibling file, validates the archive, and then atomically replaces the destination. A failed save should therefore not partially overwrite a previous valid project.

13.3 Project locks and read-only mode

Opening a project creates a sibling `.lock` file containing process and host information. If another instance already holds the lock, CurveMole opens the project read-only. Use **Save project as** to create an editable copy. A stale lock left by an abnormal termination may need manual inspection and removal after confirming that no other instance is using the project.

13.4 Normal save and portable copy

Save project updates the current project path. **Save portable copy** writes a complete separate `.fitproj` without changing the active project path. Since project data are already embedded, the portable copy is intended for sharing or archiving a snapshot.

Projects are not encrypted. Use suitable filesystem permissions or encrypted storage for confidential data.

13.5 Autosave recovery

Every ten minutes, a modified revision is written to the operating system's CurveMole user cache. An unchanged revision does not create another recovery. The three newest valid, distinct recovery files for a project are retained.

Recovery files end in `.fitproj` and can be opened through the normal Open Project dialog if needed. Saving the project normally clears its recovery files. Preview 0.8.1 does not yet show an automatic recovery chooser at startup, so after an abnormal termination inspect the CurveMole user cache before clearing application data.

13.6 Unsaved changes

An asterisk in the window title marks a dirty project. New Project, Open Project, and Quit ask whether to save, discard, or cancel when unsaved changes exist.

14. Exporting an analysis bundle

14.1 Starting an export

Choose **File > Export analysis bundle** or press **Ctrl+E**. Export mode and export content are independent.

The existing save modes remain:

- leave both mode checkboxes clear to create a new export in the selected directory;
- **Create versioned export** to create a timestamped subdirectory;
- **Update existing CurveMole-owned files after confirmation** to update files that CurveMole can verify as belonging to the current project export.

Under **What to export?**, select the desired outputs. Only **Fit results** is checked by default.

14.2 Default fit-results file

The default export creates only `fit_results.csv`. No additional visible file and no empty subdirectory is created.

Each row represents one parameter of one model component and includes the series, curve and curve ID, component and component ID, displayed function and function ID, enabled/background state, composition operator, parameter name and full parameter path, fitted value, standard error, confidence interval, bounds, fixed/link state, unit, human-readable value with uncertainty, and derived area/FWHM where defined.

This format makes the association between every data curve, the functions used to fit it, and the fitted parameters explicit in a single table.

14.3 Optional outputs

The dialog can additionally export:

- data plus fitted-function wide CSV tables;
- a tidy CSV table for Python;
- machine-readable fit results as JSON;
- a reusable `.fitmodel`;
- a `.fitproj` project copy, optionally including full uncertainty samples;
- the main plot as PNG and/or SVG;
- compact HTML and PDF summaries;
- a full reproducibility HTML report;
- covariance and correlation matrices when available;
- residual autocorrelation diagnostics when available;
- an export README.

Directories such as `data/`, `python/`, `figures/`, `report/`, `uncertainty/`, and `diagnostics/` are created only when a selected output actually writes a file there. An option whose required fit result is unavailable does not create an empty folder.

14.4 Safe overwrite policy

New exports store their ownership list in the project export configuration rather than creating a hidden sidecar file. This allows the default export to contain truly only `fit_results.csv`. Existing `.curvemole-export.json` manifests from older versions are still read for backwards-compatible updates.

When **Update existing** is selected, CurveMole refuses to overwrite a colliding file unless it can verify that the file belongs to the current project export. Unrelated files in the directory are preserved.

14.5 Wide data tables

Optional wide CSV files are intended for direct inspection and tools such as Origin or QtiPlot. Depending on data and model, columns include:

- x and y;
- total fit;
- residual as data minus fit;
- total background;
- every enabled component;
- `sigma_y` or point weight;
- mask flag.

Rows remain aligned with the complete curve, including masked and invalid entries.

14.6 Tidy data and JSON

`python/data_tidy.csv` stores one quantity per row with series, curve, component, x, value, mask, and validity fields. `python/results.json` records schema version, application version, result metadata, parameters, statistics, warnings, and settings without duplicating large numeric arrays.

14.7 Reports and reusable files

- `summary_report.html` is a compact human-readable report.
- `report/full_reproducibility.html` includes project metadata.
- `report/summary.pdf` contains a summary plot and statistics.
- `.fitmodel` stores the first project model and custom formulas for programmatic reuse.
- `.fitproj` is a complete portable project snapshot.

The exported report PDF is an analysis summary, not the CurveMole software manual.

15. Command-line interface

15.1 General form

```
curvemole [--version] [--json] [--verbose] COMMAND [OPTIONS]
```

Use `--json` for machine-readable output. Normal data go to standard output; logs and errors go to standard error.

15.2 Open the GUI

```
curvemole gui
```

The desktop executable can also receive a project or data path as its first argument.

15.3 List available functions

```
curvemole functions list
curvemole functions list --json
```

15.4 Fit one file

```
curvemole fit spectrum.csv \
  --x wavenumber \
  --y absorbance \
  --function gaussian \
  --background linear \
  --output spectrum.fitproj
```

`--x` and `--y` accept a column name or zero-based index. Optional initial values are `--center`, `--width`, and `--area`. The meaning of `--width` follows the selected function: Gaussian and Voigt use `sigma`, Lorentzian uses `gamma`, and pseudo-Voigt uses `fwhm`.

If no initial peak override is supplied, the CLI uses the strongest automatic positive peak suggestion when available. Background choices are none, constant, or linear.

15.5 Fit a series

```
curvemole fit-series scan_01.csv scan_02.csv scan_03.csv \
  --x 0 --y 1 \
  --function pseudo_voigt \
  --background constant \
  --mode sequential \
  --output series.fitproj
```

Modes are `independent`, `sequential`, and `global`. Use `--global-search` to request Differential Evolution, but remember that the simple fit CLI does not provide bound arguments. For bounded global searches, use a YAML workflow or the Python API.

15.6 Inspect and validate

```
curvemole inspect analysis.fitproj --json
curvemole inspect raw_data.dat --json
curvemole validate analysis.fitproj
curvemole validate workflow.yml
```

Inspecting a data file reports detected parsing and a preview. Validating a project checks archive structure and checksums. Validating YAML checks the supported workflow schema and required top-level structures.

15.7 Export an existing project

```
curvemole export analysis.fitproj exported_analysis
```

Use `--versioned` for a timestamped subdirectory. Use `--force` only when updating a directory whose CurveMole ownership manifest permits the overwrite.

15.8 Exit codes

Code	Meaning
0	Success
2	Command usage error
3	Data validation error
4	Fit error
5	Project format error
6	Plugin or other CurveMole domain error
10	Unexpected failure

16. Reproducible YAML workflows

16.1 Running a workflow

```
curvemole run examples/gaussian_workflow.yml --json
```

Relative paths are resolved from the workflow file's directory. Schema version 1 supports imports, custom functions, explicitly trusted plugins, models, constraints, fit settings, project output, and bundle export.

16.2 Complete example

```yaml
schema_version: 1
name: Gaussian workflow example
series_name: Synthetic spectra

imports:
  - path: gaussian.csv
    config:
      delimiter: ","
      decimal: "."
      header: true
    columns:
      x: x
      y: [y]
      sigma_y: sigma_y
    aliases: [Synthetic Gaussian]

models:
  - curve: Synthetic Gaussian
    components:
      - function: constant
        name: Background
        parameters:
          offset: 0.0
      - function: gaussian
        name: Main band
        parameters:
          area: 3.0
          center: 0.3
          sigma: 1.0
        constraints:
          sigma:
            minimum: 0.001
            maximum: 5.0

fit:
  mode: independent
  curves: [Synthetic Gaussian]
  settings:
    solver: local
    loss: linear
```

```yaml
    seed: 1729

export:
  project: gaussian_result.fitproj
  bundle: gaussian_export
  versioned: false
  overwrite: false
```

16.3 Custom formulas in YAML

```yaml
custom_functions:
  - identifier: exponential_decay
    display_name: Exponential decay
    kind: generic
    formula: offset + amplitude * exp(-x / tau)
    defaults:
      offset: 0.0
      amplitude: 1.0
      tau: 1.0
    bounds:
      tau: [0.000001, 1000000.0]
    derived:
      lifetime: tau
```

Use the custom identifier in a model component exactly like a built-in function.

16.4 Trusted plugins

List local plugin manifest paths under `plugins`. Unattended execution also requires one `--trust-plugin IDENTIFIER` argument for each plugin whose Python source may run:

```
curvemole run workflow.yml \
  --trust-plugin org.example.lineshape
```

Trust is never inferred merely because a manifest is present.

17. Python API

17.1 Stable package-level objects

The package root exports GUI-independent classes including `Curve`, `Series`, `Dataset`, `Component`, `Model`, `Parameter`, `Project`, `FitSettings`, `FitPlan`, `FitResult`, `Fitter`, `FunctionRegistry`, and `default_registry`.

17.2 Minimal fit

```
import numpy as np

from curvemole import Component, Curve, FitSettings, Fitter, Model

x = np.linspace(-5.0, 5.0, 1001)
y = 0.1 + 3.0 * np.exp(-0.5 * ((x - 0.4) / 0.8) ** 2)

curve = Curve("Synthetic band", x, y)
model = Model("One Gaussian")
model.add(Component.create("constant", initial={"offset": 0.1}))
model.add(
    Component.create(
        "gaussian",
        initial={"area": 6.0, "center": 0.3, "sigma": 1.0},
    )
)

result = Fitter().fit_single(curve, model, FitSettings(loss="linear"))
print(result.success)
print(result.statistics)
```

The Gaussian expression used to generate y above is height-parameterized, while the CurveMole Gaussian component is area-parameterized. This is why the initial model `area` is not simply the synthetic height.

17.3 Bounds and fixed values

```
peak = model.components[1]
peak.parameters["center"].minimum = -1.0
peak.parameters["center"].maximum = 1.0
peak.parameters["sigma"].minimum = 0.05
peak.parameters["sigma"].maximum = 3.0
peak.parameters["area"].fixed = False

for parameter in peak.parameters.values():
    parameter.validate()
```

17.4 Cross-curve links

```
source_path = model_a.parameter_path(curve_a.id, peak_a.id, "center")
peak_b.parameters["center"].link = "${" + source_path + "} + 5.0"
```

Construct a Global `FitPlan` containing both curve IDs. CurveMole validates the complete graph before optimization.

17.5 Global fit

```
from curvemole.core.fitting import FitMode

plan = FitPlan(
    [curve_a.id, curve_b.id],
    mode=FitMode.GLOBAL,
    settings=FitSettings(loss="linear"),
    spectrum_weights={curve_a.id: 1.0, curve_b.id: 1.0},
    equal_contribution=True,
)

result = Fitter().fit(
    plan,
    {curve_a.id: curve_a, curve_b.id: curve_b},
    {curve_a.id: model_a, curve_b.id: model_b},
)
```

17.6 API compatibility during Preview

The package root is intentionally smaller and more stable than internal modules, but version 0.x may still introduce incompatible changes. Pin the exact CurveMole version in automated research workflows and record it with results.

18. Function Builder and plugins

18.1 Formula Builder

Open **Tools > Function Builder**. Enter:

- an identifier using letters, numbers, and underscores;
- a display name;
- classification as peak, background, or generic;
- a formula in x;
- optional formulas for derived area and FWHM.

CurveMole detects every non-function symbol other than x as a parameter. Newly detected parameters default to 1.0 and are unbounded. After adding the function to the project library, add a component and edit its initial values and bounds in the normal parameter table.

Example:

```
offset + amplitude * exp(-x / tau)
```

Detected parameters are `amplitude`, `offset`, and `tau`.

18.2 Custom function persistence

Formula definitions are stored in the project. When the project reopens, CurveMole reconstructs and validates the restricted expression. A malformed custom function is skipped and recorded in the Log rather than executed.

18.3 Local plugin structure

A local plugin consists of a manifest ending in `.curvemole-plugin.json` and a Python module in the same directory. Example manifest:

```
{
  "identifier": "org.example.my-lineshape",
  "version": "1.0.0",
  "api_compatibility": "1",
  "licence": "GPL-3.0-or-later",
  "capabilities": ["functions"],
  "module": "my_lineshape.py"
}
```

The module must expose `register(registry)` and may register `FunctionDefinition` objects. Full details are in plugins.md.

18.4 Trust boundary

Formula Builder expressions are interpreted safely. Plugins are different: a plugin is Python code and can perform any action allowed to the current user account.

CurveMole reads manifests before executing a module and requires an explicit trust decision. Review source, origin, license, and version before trusting. Do not trust a plugin merely because its identifier resembles a known package.

18.5 Plugin Manager

Open **Tools > Plugin Manager**, choose a directory, and scan. Select a candidate to review its metadata, then explicitly trust and load it. Trusted identifiers and the plugin directory are stored with project UI state. Trust should be reconsidered when plugin source or version changes.

19. Troubleshooting

19.1 The application does not start on Linux

Run the AppImage from a terminal to capture the exact error. If Qt reports missing EGL, OpenGL, XKB, or XCB libraries, install the distribution packages listed in Section 3.3. If FUSE mounting fails, use `APPIMAGE_EXTRACT_AND_RUN=1`.

The AppImage is built on Ubuntu 22.04 for broad glibc compatibility. Very old Linux distributions may still require a newer runtime.

19.2 Quick Guide, manual, updates, or issue links do not open

CurveMole attempts to launch the host operating system's opener without exposing the AppImage's private Qt and C++ libraries to it. If the opener still fails, the dialog shows the path or URL to copy manually.

Use these direct addresses:

```
https://github.com/SebRoLENS/curvemole
https://github.com/SebRoLENS/curvemole/releases
https://github.com/SebRoLENS/curvemole/issues
```

19.3 Windows SmartScreen or macOS Gatekeeper warns

The Preview executables are not yet commercially signed or notarized. Confirm that the file came from the official release, verify its SHA-256 checksum, and use the operating system's documented override only if you accept the risk.

19.4 Import produces one column

The selected delimiter is probably wrong. Reopen import and compare whitespace, comma, semicolon, tab, or pipe. A decimal comma file commonly uses semicolon as its delimiter. Confirm the header choice and preview before mapping.

19.5 Imported points disappear from fitting

Inspect the Worksheet. A point is excluded when x or y is not finite, `sigma_y` is invalid, a weight is invalid, or the point is masked. Faded markers indicate masks. Use Unmask or undo the transformation that generated missing values.

19.6 A peak appears at the wrong height after placement

Built-in peaks use integrated area, not height. The graphical initializer estimates height relative to the existing model and converts it to area. For overlapping peaks or a poor background, adjust the center target, width handle, and area value before fitting.

19.7 A spline looks unstable

Use fewer, well-spaced nodes. Place nodes where background information exists rather than at every data feature. Remember that every node y value is a fit parameter by default. Fix or bound nodes when the data do not independently determine them.

19.8 Differential Evolution requests finite bounds

Set a finite lower and upper bound for every free parameter. A linked parameter does not need an independent search bound, but its resulting value must satisfy its own bounds. If defensible bounds are unavailable, improve initial estimates and use the local solver.

19.9 The fit is underdetermined or covariance is unavailable

Reduce the number of free parameters, fix known values, add justified links, widen the data range, or improve signal information. More optimizer evaluations cannot create information absent from the data.

19.10 A parameter remains at a bound

First verify the initial value and physical bound. Then inspect correlations and profile likelihood. A bound-active estimate often has an asymmetric uncertainty and may indicate that the data do not locate the optimum within the allowed region.

19.11 A sequential fit pauses

The failed curve becomes active. Inspect its initial values, masks, and constraints. Correct the model and choose **Continue paused sequence**. If component structures do not match across curves, Copy fit may be more predictable than relying on positional value transfer.

19.12 A project opens read-only

Another instance may hold the sibling `.lock` file. Close the other instance, or use Save As. Remove a lock manually only after confirming it is stale.

19.13 Export refuses to overwrite

Choose a new directory or a versioned export. If updating an existing bundle, keep its `.curvemole-export.json` manifest and explicitly enable update. CurveMole will not claim ownership of unrelated colliding files.

19.14 A resampling analysis has many failed replicates

The baseline may be near bounds, poorly initialized, or structurally unstable under perturbation. Increase robustness of the model before increasing replicate count. Report completed and failed counts with any interval.

19.15 Reporting a problem

Use GitHub Issues. Include:

- CurveMole version;
- operating system and package type;
- exact steps to reproduce;
- expected and observed behavior;
- terminal output or traceback;
- a minimal non-confidential data file if possible.

Review every attachment. Diagnostic material must not contain unpublished spectra, personal information, credentials, or proprietary data unless you intentionally make them public.

20. Reproducible research, privacy, and citation

20.1 Recommended research record

Archive at least:

- exact CurveMole version and DOI;
- original data and import mapping;
- complete `.fitproj`;
- analysis bundle;
- model functions and component order;
- initial values, bounds, fixed values, and links;
- masks and transformations;
- point uncertainties or weights;
- fit mode, spectrum weights, solver, and loss;
- random seed and replicate counts for uncertainty analyses;
- warnings, failed replicates, and residual diagnostics.

20.2 Local processing and privacy

CurveMole performs fitting locally. It has no telemetry and does not automatically upload data, projects, models, results, or formulas. Opening GitHub documentation, checking releases, reporting an issue, or using Zenodo is an explicit external action.

Plugins run locally with the user's permissions and can violate these expectations; trust only reviewed plugin code.

20.3 Citation

If CurveMole contributes to published work, cite the exact version used. The release DOI is inserted into `CITATION.cff` and the repository README after Zenodo archival. Until archival completes, the versioned GitHub release is the authoritative record:

> Romi, S. (2026). *CurveMole: Modular Scientific Curve Fitting* (Version 0.8.1) [Computer software]. GitHub. https://github.com/SebRoLENS/curvemole/releases/tag/v0.8.1

The repository provides **Cite this repository** from `CITATION.cff`.

20.4 Author and contact

Sebastiano Romi
European Laboratory for Non-Linear Spectroscopy (LENS)
University of Florence (UNIFI)
romi@lens.unifi.it

20.5 License

CurveMole is released under GPL-3.0-or-later. User data, private projects, results, and unpublished formulas remain under the user's control. Publicly distributed extensions must follow their applicable license obligations.

Appendix A. Keyboard shortcuts

Shortcuts use the platform’s standard key sequence where applicable.

Shortcut	Action
Ctrl+N	New project
Ctrl+O	Open project
Ctrl+I	Import data
Ctrl+S	Save project
Ctrl+Shift+S	Save project as on common desktop mappings
Ctrl+E	Export analysis bundle
Ctrl+Z	Undo
Standard Redo shortcut	Redo
Ctrl++	Add component
F5	Open Fit dialog
F1 or platform Help shortcut	Quick Start
Esc	Cancel graphical peak or spline placement
Ctrl while dragging	Change a fixed graphical parameter without unfixing it
Right-drag on plot	Mask or unmask an interval directly

Appendix B. Built-in identifiers

Identifier	Display name	Parameters
`gaussian`	Gaussian	`area`, `center`, `sigma`
`lorentzian`	Lorentzian	`area`, `center`, `gamma`
`voigt`	Voigt	`area`, `center`, `sigma`, `gamma`
`pseudo_voigt`	Pseudo-Voigt	`area`, `center`, `fwhm`, `eta`
`constant`	Constant background	`offset`
`linear`	Linear background	`intercept`, `slope`
`polynomial`	Polynomial background	`c0` through selected order
`cubic_spline`	Cubic-spline background	`y0` through last node

Appendix C. File extensions

Extension	Purpose
`.fitproj`	Complete CurveMole project
`.fitmodel`	Reusable model and formula definitions
`.yml`, `.yaml`	Reproducible workflow
`.txt`, `.dat`, `.csv`, `.tsv`	Imported one-dimensional data
`.whl`	Installable Python package
`.AppImage`	Linux desktop package
`.exe`	Windows desktop executable
`.dmg`	macOS disk image

Appendix D. Preview 0.8.1 limitations

The following boundaries are important when evaluating this release:

- scientific validation toward 1.0.0 is still in progress;
- only nonlinear least squares is implemented as the final local optimizer;
- `sigma_x` is stored but not used in optimization;
- the GUI does not yet provide a dedicated parameter-path picker for complex links;
- fit ranges exist in the core model but do not yet have a complete graphical editor;
- the GUI does not yet provide one consolidated table for every fit statistic;
- autosave recovery exists, but there is no automatic startup recovery chooser;
- desktop packages are unsigned;
- the interface and manual are currently maintained in English;
- plugin API and other 0.x APIs may change before 1.0.0.

These limitations are stated so that testing can focus on genuine behavior rather than inferred promises. Report unclear workflows and reproducible defects through the issue tracker.

Appendix E. Building this manual

The Markdown file you are reading is the only hand-edited manual source. From a source checkout with Pandoc, XeLaTeX, and `latexmk` installed, run:

```
python scripts/build_manual.py
```

The command validates version declarations and local links, then creates:

```
docs/CurveMole_User_Manual.tex
docs/CurveMole_User_Manual.pdf
```

GitHub Actions repeats this process for documentation changes and releases. Release assets use versioned filenames and include both LaTeX and PDF editions. Do not edit the generated files directly; changes will be replaced by the next automated build.

Background subtraction and spline controls

Use **Data > Subtract background...** when the measured zero line should be corrected before or after model construction. Two methods are available:

- **Constant from x interval** asks for an x interval, calculates the median y value in that interval, and subtracts that constant from the entire active curve. The interval calculation deliberately includes masked data points if they lie inside the requested x range.
- **Spline from graph** starts the graphical spline editor. Left-click adds a node, right-click removes the nearest node, and a left double-click or **Finish** accepts the spline. Nodes may be placed inside masked regions. The resulting spline is evaluated and subtracted over the entire x array, including masked regions.

Background subtraction is stored as a reversible data transformation. The original imported arrays are retained, **Undo** reverses the subtraction, and **Restore original data** in the Data Calculator removes the transformation history. Masks are not removed or changed by background subtraction: they still control which data points participate in fitting.

For cubic-spline model components, the y values of newly placed spline nodes are **fixed by default**. This prevents an intentionally drawn baseline from drifting when a fit starts. Individual nodes can be unlocked with the **Fixed** checkbox in the parameter table. The **Lock all** and **Unlock all** controls below the parameter table change the fixed state of every parameter in the selected component at once. The x positions of spline nodes remain fixed; unlocking a node allows its y value to vary or be dragged.

A spline is evaluated continuously through masked regions even though masked measurements are excluded from the objective function. This makes it possible to define a baseline through masked peaks or artifacts without temporarily unmasking those data.